Computer Networks: Transport and Application Layers, Network Security

Contents

Computer Networks: Transport and Application Layers, Network Security **1**

1. Transport Layer Fundamentals . . . 2
    - 1.1 Transport Layer Services . . . 2
    - 1.2 Multiplexing and Demultiplexing . . . 2
    - 1.3 UDP: Connectionless Transport . . . 3
2. Reliable Data Transfer . . . 3
    - 2.1 Principles and Protocol Design . . . 3
    - 2.2 Pipelined Protocols . . . 4
3. TCP: Connection-Oriented Transport . . . 5
    - 3.1 TCP Overview and Segment Structure . . . 5
    - 3.2 TCP Reliability Mechanisms . . . 6
    - 3.3 TCP Flow Control . . . 6
    - 3.4 TCP Connection Management . . . 7
4. Congestion Control . . . 7
    - 4.1 Principles of Congestion Control . . . 7
    - 4.2 TCP Congestion Control Algorithms . . . 8
    - 4.3 Modern Developments . . . 9
5. Application Layer Fundamentals . . . 10
    - 5.1 Network Application Principles . . . 10
    - 5.2 Transport Service Requirements . . . 11
6. Web and HTTP . . . 12
    - 6.1 HTTP Protocol Basics . . . 12
    - 6.2 HTTP State Management and Optimization . . . 13
    - 6.3 HTTP Evolution . . . 14
7. Electronic Mail . . . 14
8. Domain Name System . . . 15
    - 8.1 DNS Architecture and Services . . . 15
    - 8.2 DNS Operation and Security . . . 16
9. Network Security Fundamentals . . . 17
    - 9.1 Security Concepts and Threat Model . . . 17
    - 9.2 Cryptographic Foundations . . . 17
10. Authentication and Message Integrity . . . 19
    - 10.1 Authentication Protocols . . . 19
    - 10.2 Digital Signatures and Certification . . . 20
11. Securing Network Protocols . . . 21
    - 11.1 Secure Email . . . 21
    - 11.2 Transport Layer Security (TLS) . . . 21
    - 11.3 Network Layer Security (IPsec) . . . 22
    - 11.4 Wireless and Mobile Security . . . 23
12. Operational Security . . . 24
13. Video Streaming and Content Distribution Networks . . . 26
    - 13.1 Video Streaming Fundamentals . . . 26
    - 13.2 Content Distribution Networks . . . 27
14. Socket Programming . . . 27
    - 14.1 Socket Programming Fundamentals . . . 27
    - 14.2 UDP Socket Programming . . . 27
    - 14.3 TCP Socket Programming . . . 28

License and Attribution . . . 28
- License . . . 28
- Attribution . . . 28
- Disclaimer . . . 29

1. Transport Layer Fundamentals

1.1 Transport Layer Services

The transport layer provides logical communication between application processes running on different hosts. **Transport protocols execute in end systems, not in network routers**. The sender breaks application messages into segments and passes them to the network layer, while the receiver reassembles segments into messages and passes them to the application layer.

The Internet offers two transport protocols to applications: TCP and UDP. These protocols differ fundamentally in their service models and guarantees.

Transport vs. Network Layer

The network layer provides communication between hosts, while the **transport layer provides communication between processes**. The transport layer relies on and enhances network layer services. An analogy: hosts are houses, processes are children in those houses, application messages are letters in envelopes, the transport protocol is like household members who distribute mail to the correct recipient, and the network layer protocol is the postal service.

Sockets

Processes send and receive messages through sockets. A socket functions analogously to a door: the sending process pushes the message out the door and relies on transport infrastructure to deliver it to the socket at the receiving process. Two sockets are involved in any communication: one on each side.

Transport Layer Actions

At the sender, the transport layer receives an application-layer message, determines segment header field values, creates the segment, and passes it to IP. At the receiver, the transport layer receives the segment from IP, checks header values, extracts the application-layer message, and demultiplexes the message up to the application via the socket.

TCP vs. UDP Services

TCP provides reliable, in-order delivery with congestion control, flow control, and connection setup. UDP offers unreliable, unordered delivery as a no-frills extension of best-effort IP. Neither protocol provides delay guarantees or bandwidth guarantees.

1.2 Multiplexing and Demultiplexing

Multiplexing at the sender involves handling data from multiple sockets and adding transport headers for later demultiplexing. Demultiplexing at the receiver uses header information to deliver received segments to the correct socket.

How Demultiplexing Works

The host receives IP datagrams, each with source and destination IP addresses. Each datagram carries one transport-layer segment containing source and destination port numbers. The host uses IP addresses and port numbers to direct the segment to the appropriate socket.

Connectionless Demultiplexing (UDP)

When creating a UDP socket, the application must specify a host-local port number. When a receiving host receives a UDP segment, it checks the destination port number and directs the segment to the socket with that port number. UDP datagrams with the same destination port but different source IP addresses or source port numbers are directed to the same socket at the receiving host.

Connection-Oriented Demultiplexing (TCP)

A TCP socket is identified by a 4-tuple: source IP address, source port number, destination IP address, and destination port number. A server may support many simultaneous TCP sockets, each identified by its own 4-tuple and associated with a different connecting client. The receiver uses all four values to direct the segment to the appropriate socket.

Clarifications

UDP demultiplexing uses only the destination port number. UDP does not establish connections with individual senders; all datagrams arriving on the same destination port are delivered to the same socket. Source and destination IP addresses are present in the IP header but are used only for packet delivery and replies, not for socket selection.

TCP demultiplexing is connection-oriented. The kernel identifies each connection using the full 4-tuple. A UDP socket is an endpoint identified by IP address, port number, and protocol (UDP). It has the same structural identification as a TCP socket but different behavior.

Key differences from TCP: UDP has no connection setup (no SYN/SYN-ACK/ACK handshakes), no concept of established sockets, and uses a single socket for all clients. The server simply binds a socket to a port and receives datagrams. TCP creates separate connection sockets for each client, while UDP uses one socket for all clients sending datagrams to that port.

1.3 UDP: Connectionless Transport

UDP is a bare bones Internet transport protocol with best effort service. UDP segments may be lost or delivered out of order to the application. It is connectionless: no handshaking between sender and receiver, and each segment is handled independently.

Why UDP Exists

UDP has several advantages: no connection establishment (avoiding RTT delay), simple design with no connection state at sender or receiver, small header size, and no congestion control allowing it to transmit as fast as desired. UDP can function in the face of congestion.

UDP Use Cases

UDP is used for streaming multimedia applications (loss tolerant, rate sensitive), DNS, SNMP, and HTTP/3. If reliable transfer is needed over UDP (such as in HTTP/3), reliability must be added at the application layer, along with congestion control at the application layer.

UDP Transport Layer Actions

The UDP sender receives an application-layer message, determines UDP segment header field values, creates the UDP segment, and passes it to IP. The UDP receiver receives the segment from IP, checks the UDP checksum header value, extracts the application-layer message, and demultiplexes the message up to the application via the socket.

UDP Segment Header

The UDP segment format includes source port number, destination port number, length (in bytes of the UDP segment including header), checksum, and application data (payload).

UDP Checksum

The goal of the checksum is to detect errors (flipped bits) in the transmitted segment. The sender treats the contents of the UDP segment (including UDP header fields and IP addresses) as a sequence of 16-bit integers. The checksum is the one's complement sum of segment content, and the checksum value is placed into the UDP checksum field.

The receiver computes the checksum of the received segment and checks if it equals the checksum field value. If they are not equal, an error is detected. If they are equal, no error is detected, though errors may still be present. When adding numbers, a carryout from the most significant bit must be added to the result.

The Internet checksum provides weak protection: even when numbers change due to bit flips, the checksum may remain unchanged.

UDP Summary

UDP is a no-frills protocol where segments may be lost or delivered out of order. It provides best effort service with a send-and-hope-for-the-best approach. UDP advantages include no setup or handshaking needed (no RTT incurred), ability to function when network service is compromised, and basic reliability support through checksums. Additional functionality can be built on top of UDP in the application layer.

---

2. Reliable Data Transfer

2.1 Principles and Protocol Design

Reliable data transfer is a critical service abstraction where the sending process delivers data to the receiving process over a reliable channel. The implementation uses an unreliable channel at the network layer, with sender-side and receiver-side reliable data transfer protocols.

The complexity of the reliable data transfer protocol depends strongly on the characteristics of the unreliable channel: whether it loses, corrupts, or reorders data. The sender and receiver do not know the state of each other unless communicated via messages.

Reliable Data Transfer Protocol Interfaces

The protocol has four key functions: `rdt_send()` is called from above (by the application) and passes data to deliver to the receiver upper layer. `udt_send()` is called by rdt to transfer the packet over the unreliable channel to the receiver. `rdt_rcv()` is called when a packet arrives on the receiver side of the channel. `deliver_data()` is called by rdt to deliver data to the upper layer. The protocol supports bidirectional communication over an unreliable channel.

Protocol Development Approach

Protocols are developed incrementally, considering only unidirectional data transfer (though control information flows in both directions). Finite state machines (FSM) specify sender and receiver behavior. When in a particular state, the next state is uniquely determined by the next event, which causes a state transition and associated actions.

rdt1.0: Reliable Transfer Over Reliable Channel

When the underlying channel is perfectly reliable (no bit errors, no packet loss), separate FSMs specify sender and receiver behavior. The sender sends data into the underlying channel, and the receiver reads data from the underlying channel.

rdt2.0: Channel With Bit Errors

When the underlying channel may flip bits in packets, a checksum (such as the Internet checksum) detects bit errors. Recovery from errors uses acknowledgements (ACKs) where the receiver explicitly tells the sender the packet was received correctly, and negative acknowledgements (NAKs) where the receiver explicitly tells the sender the packet had errors. The sender retransmits the packet upon receipt of a NAK. This is a stop-and-wait protocol: the sender sends one packet, then waits for receiver response.

The receiver's state (whether it correctly received the message) is not known to the sender unless communicated from receiver to sender, which is why the protocol is necessary.

rdt2.0 Fatal Flaw

If ACK or NAK messages are corrupted, the sender doesn't know what happened at the receiver and cannot simply retransmit due to possible duplicates.

rdt2.1: Handling Garbled ACK/NAKs

To handle duplicates, the sender retransmits the current packet if ACK/NAK is corrupted, adds a sequence number to each packet, and the receiver discards duplicate packets without delivering them up. With stop-and-wait, two sequence numbers (0 and 1) suffice.

The sender must check if received ACK/NAK is corrupted, requiring twice as many states. The state must remember whether the expected packet should have sequence number 0 or 1. The receiver must check if the received packet is a duplicate; the state indicates whether packet sequence number 0 or 1 is expected. The receiver cannot know if its last ACK/NAK was received correctly at the sender.

rdt3.0: Channels With Errors and Loss

When the underlying channel can also lose packets (data or ACKs), checksums, sequence numbers, ACKs, and retransmissions help but are not sufficient. The sender waits a reasonable amount of time for an ACK. If no ACK is received within this time, the sender retransmits. If a packet or ACK is delayed but not lost, retransmission creates a duplicate, but sequence numbers already handle this. The receiver must specify the sequence number of the packet being ACKed. A countdown timer interrupts after a reasonable timeout interval.

Performance of rdt3.0

Stop-and-wait operation has poor performance. For a 1 Gbps link with 15 ms propagation delay and an 8000 bit packet, transmission delay is 8 microseconds. Sender utilization (fraction of time sender is busy sending) is extremely low: 0.00027 for this example. The protocol limits the performance of the underlying infrastructure.

2.2 Pipelined Protocols

Pipelining allows the sender to have multiple in-flight, yet-to-be-acknowledged packets. This requires an increased range of sequence numbers and buffering at sender and/or receiver. Three-packet pipelining increases utilization by a factor of 3.

Go-Back-N Protocol

The sender maintains a window of up to N consecutive transmitted but unacknowledged packets. Sequence numbers use k-bit values in the packet header. Cumulative ACK is used: ACK(n) acknowledges all packets up to and including sequence number n. Upon receiving ACK(n), the sender moves the window forward to begin at n+1. A timer is maintained for the oldest in-flight packet. On timeout(n), the sender retransmits packet n and all higher sequence number packets in the window.

The receiver uses ACK-only: it always sends an ACK for the correctly-received packet with the highest in-order sequence number. This may generate duplicate ACKs. The receiver only needs to remember rcv_base. Upon receipt of an out-of-order packet, the receiver can discard it (or buffer it as an implementation decision) and re-ACK the packet with the highest in-order sequence number.

Selective Repeat Protocol

Selective repeat uses pipelining with multiple packets in flight. The receiver individually acknowledges all correctly received packets and buffers packets as needed for in-order delivery to the upper layer.

The sender conceptually maintains a timer for each unacknowledged packet. On timeout, it retransmits only the single unacknowledged packet associated with the timeout. The sender conceptually maintains a window over N consecutive sequence numbers, limiting pipelined in-flight packets to be within this window.

For data from above, if the next available sequence number is in the window, the sender sends the packet. On timeout(n), the sender resends packet n and restarts the timer. When ACK(n) is received in the range [sendbase, sendbase+N-1], the sender marks packet n as received. If n is the smallest unacknowledged packet, the sender advances the window base to the next unacknowledged sequence number.

The receiver handles packets in [rcvbase, rcvbase+N-1] by sending ACK(n). Out-of-order packets are buffered, and in-order packets are delivered (along with buffered in-order packets), advancing the window to the next not-yet-received packet. For packets in [rcvbase-N, rcvbase-1], the receiver sends ACK(n). Otherwise, the receiver ignores the packet.

Reliable Data Transfer Mechanisms

Key mechanisms for reliable data transfer include error detection (checksums), ACKs and NAKs, retransmission, sequence numbers for duplicate detection, and timeout-and-retransmit for lost messages.

---

3. TCP: Connection-Oriented Transport

3.1 TCP Overview and Segment Structure

TCP is defined in RFCs 793, 1122, 2018, 5681, and 7323. It is point-to-point (one sender, one receiver) and provides reliable, in-order byte stream delivery with no message boundaries. TCP is full duplex, supporting bidirectional data flow in the same connection. It is connection-oriented, requiring handshaking to initialize sender and receiver state before data exchange. TCP implements flow control so the sender will not overwhelm the receiver. Pipelining is controlled by TCP congestion and flow control, which set the window size. The maximum segment size (MSS) is a key parameter.

Why TCP Uses a Byte Stream

TCP provides reliable, ordered delivery, making it easier to treat data as a continuous stream of bytes rather than discrete messages. Applications can write data in arbitrary chunks without message boundaries. TCP is free to segment, coalesce, retransmit, and reorder bytes as needed. This enables partial retransmissions (only missing bytes) and maximizes efficiency for MSS selection, congestion control, Nagle's algorithm, delayed ACKs, and other optimizations.

How TCP Manages Sequencing

TCP uses byte-oriented sequencing, not segment-oriented. The sequence number field holds the byte number of the first byte in the segment. ACKs are cumulative, indicating the next byte expected. Out-of-order data is buffered, and the receiver continues sending the same ACK (duplicate ACKs). Three duplicate ACKs trigger fast retransmit of the missing bytes.

TCP Segment Structure

The TCP segment includes source port number, destination port number, sequence number (byte stream number of first byte in segment's data), acknowledgment number (sequence number of next byte expected from the other side, providing cumulative ACK), header length, flags (including RST, SYN, FIN for connection management; C, E for congestion notification; U, A, P for

data handling), receive window (for flow control, indicating number of bytes receiver is willing to accept), checksum, urgent data pointer, options, and application data.

The TCP specification does not mandate how receivers handle out-of-order segments; this is an implementation decision.

3.2 TCP Reliability Mechanisms

Round Trip Time and Timeout

TCP must set timeout values longer than RTT, but RTT varies. If the timeout is too short, premature timeouts cause unnecessary retransmissions. If too long, the reaction to segment loss is slow.

RTT is estimated using SampleRTT, measured from segment transmission until ACK receipt (ignoring retransmissions). SampleRTT varies, so an estimated RTT is computed by averaging several recent measurements rather than using just the current SampleRTT.

EstimatedRTT uses an exponential weighted moving average (EWMA):

$$EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT$$

Where the influence of past samples decreases exponentially fast. A typical value is $\alpha = 0.125$.

The timeout interval is EstimatedRTT plus a safety margin. Larger variation in EstimatedRTT requires a larger safety margin:

$$TimeoutInterval = EstimatedRTT + 4 \times DevRTT$$

DevRTT is the EWMA of SampleRTT deviation from EstimatedRTT:

$$DevRTT = (1 - \beta) \times DevRTT + \beta \times \mid SampleRTT - EstimatedRTT \mid$$

With typical $\beta = 0.25$.

TCP Sender Operation

When data is received from the application, the sender creates a segment with a sequence number (the byte-stream number of the first data byte in the segment) and starts a timer if not already running. The timer is conceptually for the oldest unacknowledged segment with expiration interval TimeoutInterval.

On timeout, the sender retransmits the segment that caused the timeout and restarts the timer. When an ACK is received and it acknowledges previously unacknowledged segments, the sender updates what is known to be ACKed and starts the timer if there are still unacknowledged segments.

TCP Receiver ACK Generation

The receiver follows specific rules: For an in-order segment with expected sequence number where all data up to expected sequence number is already ACKed, the receiver uses delayed ACK (waiting up to 500ms for the next segment; if no next segment arrives, it sends an ACK). For an in-order segment with expected sequence number when one other segment has an ACK pending, the receiver immediately sends a single cumulative ACK, acknowledging both in-order segments. For an out-of-order segment with higher-than-expected sequence number indicating a gap, the receiver immediately sends a duplicate ACK indicating the sequence number of the next expected byte. For a segment that partially or completely fills a gap, the receiver immediately sends an ACK, provided the segment starts at the lower end of the gap.

TCP Fast Retransmit

Receipt of three duplicate ACKs indicates that three segments were received after a missing segment, suggesting the lost segment is likely lost. If the sender receives three additional ACKs for the same data (triple duplicate ACKs), it resends the unacknowledged segment with the smallest sequence number without waiting for timeout. This is the fast retransmit mechanism.

3.3 TCP Flow Control

Flow control prevents the network layer from delivering data faster than the application layer removes data from socket buffers. If this occurs, the receiver's buffers can overflow. TCP implements receiver control of the sender so the sender will not overflow the receiver's buffer by transmitting too much too fast.

TCP Flow Control Mechanism

The TCP receiver advertises free buffer space in the rwnd field in the TCP header. RcvBuffer size is set via socket options (typical default is 4096 bytes), and many operating systems auto-adjust RcvBuffer. The sender limits the amount of unacknowledged (in-flight) data to the received rwnd value. This guarantees the receive buffer will not overflow.

3.4 TCP Connection Management

Before exchanging data, sender and receiver perform a handshake to agree to establish the connection (each knowing the other is willing to establish the connection) and agree on connection parameters such as starting sequence numbers.

Both client and server maintain connection state (ESTAB) and connection variables including sequence numbers for client-to-server and server-to-client directions, and RcvBuffer sizes at server and client.

Need for Three-Way Handshake

A two-way handshake does not always work in a network with variable delays, retransmitted messages due to message loss, message reordering, and inability to see the other side. Problems include half-open connections where no client exists and duplicate data being accepted due to retransmissions.

TCP Three-Way Handshake

The client chooses an initial sequence number x, sends a **TCP SYN** message with SYNbit=1 and Seq=x, and enters **SYN SENT** state. The server, in **LISTEN** state, chooses an initial sequence number y, sends a **TCP SYNACK** message with SYNbit=1, Seq=y, ACKbit=1, and ACKnum=x+1, and enters **SYN RCVD** state.

The client receives the SYNACK(x), which indicates the server is live. The client sends an ACK for the SYNACK with ACKbit=1 and ACKnum=y+1, enters **ESTAB** state, and this segment may contain client-to-server data. The server receives ACK(y), which indicates the client is live, and enters **ESTAB** state.

Closing a TCP Connection

Both client and server close their side of the connection by sending a TCP segment with FIN bit = 1. The receiver responds to a received FIN with an ACK. On receiving FIN, the ACK can be combined with the receiver's own FIN. Simultaneous **FIN exchanges** can be handled.

---

4. Congestion Control

4.1 Principles of Congestion Control

Congestion occurs when **too many sources send too much data too fast for the network to handle**. Manifestations include long delays (queueing in router buffers) and packet loss (buffer overflow at routers). Congestion control addresses too many senders sending too fast, which is different from flow control (one sender too fast for one receiver).

Causes and Costs of Congestion: Scenario 1

With two flows, one router with infinite buffers, and input/output link capacity R, no retransmissions are needed. The rate at which packets reach the router is λ_{in}, the maximum physical speed of the link is R (depending on router hardware), and the rate at which packets leave the router is $\lambda_{out} = \min \lambda_{in}, R$.

Maximum per-connection throughput is $R/2$. As the arrival rate approaches this limit ($\lambda_{in} \to R/2$), large delays occur due to high utilization and queueing.

Causes and Costs of Congestion: Scenario 2

With one router and finite buffers, the sender retransmits lost or timed-out packets. Application-layer input equals application-layer output: $\lambda_{in} = \lambda_{out}$. Transport-layer input includes retransmissions: $\lambda'_{out} \leq \lambda_{in}$.

With perfect knowledge where the sender only sends when router buffers are available, throughput remains at $R/2$ but some capacity is wasted due to necessary retransmissions. With some perfect knowledge where packets can be lost due to full buffers but the sender knows when packets are dropped and only resends lost packets, **wasted capacity increases due to retransmissions**.

In a realistic scenario with unnecessary duplicates, packets can be lost or dropped due to full buffers, and senders can time out prematurely, sending two copies that are both delivered. This further **wastes capacity due to unneeded retransmissions**, and some retransmissions (both needed and unneeded duplicates) are delivered.

Costs of congestion include more work (retransmission) for given receiver throughput and unneeded retransmissions causing the link to carry multiple copies of a packet, decreasing maximum achievable throughput.

Causes and Costs of Congestion: Scenario 3

With four senders, multi-hop paths, and timeout/retransmit, as λ'_{in} for one flow increases, arriving packets from other flows at the upper queue are dropped, causing their throughput to approach zero. Another cost of congestion: when a **packet is dropped**, any **upstream transmission capacity and buffering** used for that packet **was wasted**.

Congestion Control Insights

Throughput can never exceed capacity. Loss and retransmission decrease effective throughput. Delay increases as capacity is approached. Unneeded duplicates further decrease effective throughput. Upstream transmission capacity and buffering are wasted for packets lost downstream.

Approaches to Congestion Control

End-end congestion control involves no explicit feedback from the network. Congestion is inferred from observed loss and delay. This is the approach taken by TCP.

Network-assisted congestion control involves routers providing direct feedback to sending/receiving hosts with flows passing through congested routers. Routers may indicate the congestion level or explicitly set the sending rate. Protocols include TCP ECN, ATM, and DECbit.

4.2 TCP Congestion Control Algorithms

TCP AIMD (Additive Increase Multiplicative Decrease)

Senders increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss events.

Additive increase: increase sending rate by 1 maximum segment size (MSS) every RTT until loss is detected.

Multiplicative decrease: cut sending rate in half at each loss event. This creates sawtooth behavior, probing for bandwidth.

TCP cuts the *rate in half on loss detected by triple duplicate ACK* (**TCP Reno**) or *cuts to 1 MSS when loss is detected by timeout* (**TCP Tahoe**). AIMD is a *distributed, asynchronous algorithm that optimizes congested flow rates network-wide and has desirable stability properties.*

TCP Congestion Control Details

The TCP sender limits transmission: LastByteSent - LastByteAcked ≤ cwnd. The *congestion window* (cwnd) is dynamically adjusted in response to observed network congestion, implementing TCP congestion control.

TCP sending behavior roughly sends cwnd bytes, waits RTT for ACKs, then sends more bytes. *TCP rate* is approximately

$$\frac{cwnd}{RTT}\,[B/s]$$

TCP Slow Start

When a connection begins, the rate increases exponentially until the first loss event. Initially cwnd = 1 MSS. The *sender doubles cwnd every RTT by incrementing cwnd for every ACK received*. The initial rate is slow but ramps up exponentially fast.

Transition to Congestion Avoidance

The exponential *increase should switch to linear when cwnd reaches half its value before timeout*. A variable **ssthresh** (slow start threshold) is used. On a loss event, *ssthresh is set to half of cwnd just before the loss event*.

Summary: TCP Congestion Control State Machine

The protocol has three states: slow start, congestion avoidance, and fast recovery.

In *slow start*, cwnd increases exponentially: + On timeout, ssthresh = cwnd/2, cwnd = 1 MSS, dupACKcount = 0, and the missing segment is retransmitted. + On new ACK with cwnd > ssthresh, the state transitions to congestion avoidance. + On three duplicate ACKs, ssthresh = cwnd/2, cwnd = ssthresh + 3, the missing segment is retransmitted, and the state transitions to fast recovery.

In *congestion avoidance*, cwnd increases linearly: cwnd = cwnd + MSS×(MSS/cwnd): + On timeout, the same actions as in slow start occur. + On three duplicate ACKs, the state transitions to fast recovery.

In fast recovery, cwnd increases: cwnd = cwnd + MSS on each duplicate ACK, and new segments are transmitted as allowed: + On timeout, the same actions as in slow start occur. + On new ACK, cwnd = ssthresh, dupACKcount = 0, and the state transitions to congestion avoidance.

4.3 Modern Developments

TCP CUBIC

TCP CUBIC asks whether there is a better way than AIMD to probe for usable bandwidth. The insight is that W_max (the sending rate at which congestion loss was detected) indicates that the congestion state of the bottleneck link probably has not changed much. After cutting the rate/window in half on loss, the protocol initially ramps to W_max faster, then approaches W_max more slowly.

K is the point in time when TCP window size will reach W_max and is tunable. Increases are larger when further from K and smaller (cautious) when nearer K. The window increases as a function of the cube of the distance between current time and K. TCP CUBIC is the default in Linux and was the most popular TCP for Web servers until approximately 2024.

TCP and the Bottleneck Link

TCP (classic and CUBIC) increases sending rate until packet loss occurs at some router's output: the bottleneck link. The bottleneck link is almost always busy with a packet queue that is almost never empty and sometimes overflows, causing packet loss.

Understanding congestion focuses on the congested bottleneck link. Increasing TCP sending rate will not increase end-to-end throughput at the congested bottleneck, which is already transmitting at full rate R. However, increasing TCP sending rate will increase measured RTT due to increased queueing delay at the bottleneck link. The goal is to keep the end-to-end pipe just full, but not fuller.

When multiple flows collectively congest the bottleneck link, each executes its own TCP congestion control. Ideally, each of N flows should receive throughput of R/N.

Keeping the Pipe Just Full

To keep the bottleneck link busy transmitting while avoiding high delays and buffering, the measured throughput equals the number of bytes sent (in flight) divided by the RTT interval. Intuitively, the ideal amount of in-flight data for a connection equals the connection's share of bottleneck bandwidth times the minimum RTT (the connection's bandwidth-delay product).

Explicit Congestion Notification (ECN)

TCP deployments often implement network-assisted congestion control. Two bits in the IP header (ToS field) are marked by network routers to indicate congestion. The policy for marking is chosen by the network operator. Congestion indication is carried to the destination, and the destination sets the ECE bit on the ACK segment to notify the sender of congestion. This involves both IP (IP header ECN bit marking) and TCP (TCP header C, E bit marking).

TCP Fairness

The fairness goal is that if K TCP sessions share the same bottleneck link of bandwidth R, each should have an average rate of R/K. Under idealized assumptions (same RTT, fixed number of sessions, only in congestion avoidance), TCP is fair. Additive increase gives a slope of 1 as throughput increases, and multiplicative decrease decreases throughput proportionally, leading to convergence toward equal bandwidth share.

Fairness and Other Applications

Multimedia applications often do not use TCP because they do not want rate throttled by congestion control. Instead, they use UDP, sending audio/video at constant rate and tolerating packet loss. There is no enforcement mechanism policing the use of congestion control.

For fairness with parallel TCP connections, an application can open multiple parallel connections between two hosts. Web browsers do this. For a link of rate R with 9 existing connections, a new application asking for 1 TCP connection gets rate R/10, but a new application asking for 11 TCP connections gets approximately R/2.

Evolution of Transport Functionality

TCP and UDP have been the principal transport protocols for 40 years. Different flavors of TCP have been developed for specific scenarios: long fat pipes (large data transfers) with many packets in flight where loss shuts down the pipeline, wireless networks

with loss due to noisy links and mobility that TCP treats as congestion loss, long-delay links with extremely long RTTs, data center networks that are latency sensitive, and background traffic flows that are low priority.

Transport-layer functions are moving to the application layer on top of UDP, exemplified by HTTP/3 using QUIC.

QUIC: Quick UDP Internet Connections

QUIC is an application-layer protocol on top of UDP designed to increase HTTP performance. It is deployed on many Google servers and applications (Chrome, mobile YouTube app).

QUIC adopts approaches for connection establishment, error control, and congestion control. It supports multiple application-level streams multiplexed over a single QUIC connection with separate reliable data transfer and security but common congestion control. Error and congestion control algorithms parallel well-known TCP ones. Connection establishment achieves reliability, congestion control, authentication, encryption, and state establishment in one or even zero RTT.

QUIC avoids head-of-line blocking: with HTTP/2 over a single TCP connection, recovery from packet loss stalls all object transmissions. Browsers have an incentive to open multiple parallel TCP connections to reduce stalling and increase overall throughput. QUIC provides per-object error and congestion control with more pipelining over UDP. HTTP/3 adds security and per-object control over UDP.

---

5. Application Layer Fundamentals

5.1 Network Application Principles

Network applications include social media, Web, text messaging, email, multi-user network games, streaming stored video, P2P file sharing, voice over IP, real-time video conferencing, Internet search, and remote login.

Creating Network Applications

To create a network application, write programs that run on different end systems and communicate over the network. For example, web server software communicates with browser software. There is no need to write software for network-core devices, as they do not run user applications. This design allows for rapid application development and propagation.

Client-Server Paradigm

In the client-server model, the server is an always-on host with a permanent IP address, often located in data centers for scaling. Clients contact and communicate with the server, may be intermittently connected, may have dynamic IP addresses, and do not communicate directly with each other. Examples include HTTP, IMAP, and FTP.

Peer-to-Peer Architecture

In peer-to-peer architecture, there is no always-on server. Arbitrary end systems directly communicate. Peers request service from other peers and provide service in return. This offers self-scalability: new peers bring new service capacity as well as new service demands. Peers are intermittently connected and change IP addresses, making management complex. An example is P2P file sharing using BitTorrent.

Process Communication

A process is a program running within a host. Within the same host, two processes communicate using inter-process communication (defined by the OS). Processes in different hosts communicate by exchanging messages. Applications with P2P architectures have both client processes and server processes. The client process initiates communication, and the server process waits to be contacted.

Sockets

Processes send and receive messages through sockets. A socket is analogous to a door: the sending process pushes the message out the door and relies on transport infrastructure on the other side to deliver the message to the socket at the receiving process. Two sockets are involved, one on each side.

Addressing Processes

To receive messages, a process must have an identifier. A host device has a unique 32-bit IP address, but this does not suffice for identifying the process because many processes can run on the same host. The identifier includes both the IP address and port numbers associated with the process on the host. Example port numbers: HTTP server uses port 80, mail server uses port 25.

Application-Layer Protocols

An application-layer protocol defines types of messages exchanged (request, response), message syntax (what fields are in messages and how fields are delineated), message semantics (meaning of information in fields), and rules for when and how processes send and respond to messages.

Open protocols are defined in RFCs, everyone has access to protocol definitions, and they allow for interoperability. Examples include HTTP and SMTP. Proprietary protocols include Zoom.

5.2 Transport Service Requirements

Data Integrity

Some applications (file transfer, web transactions) require 100 percent reliable data transfer. Other applications (audio) can tolerate some loss. Applications requiring reliable delivery need data to arrive in order, without losses, and without corruption, requiring mechanisms for retransmissions, acknowledgments, sequence numbers, and duplicate detection. TCP provides reliable byte-stream delivery, while UDP does not provide integrity guarantees.

Timing Requirements

Some applications (Internet telephony, interactive games) require low delay to be effective. For timing-sensitive applications, low delay is more important than perfect accuracy. Retransmissions (TCP) introduce unacceptable latency. Some loss is tolerated as long as playout remains smooth. These applications typically use UDP or QUIC, implementing their own playout and loss-recovery logic.

Throughput

Some applications (multimedia) require a minimum amount of throughput to be effective. Streaming audio/video requires sustained bandwidth; high-quality streams (HD/4K) require several Mbps. If throughput drops, quality must adapt (using DASH). Elastic applications (email, Web browsing) work with whatever throughput is available. The Internet transport layer cannot guarantee throughput, but TCP provides fair congestion control among flows. TCP or QUIC is used for adaptive bitrate streaming, while UDP is used for real-time media to avoid congestion-related stalls.

Throughput Control in Streaming Services

Netflix and YouTube do not control network throughput; they measure it and adapt. The client downloads video in chunks (segments of a few seconds). Each chunk is available in multiple quality levels. For every chunk, the client measures actual download speed and selects the highest bitrate it can download safely. Each video is encoded at multiple bitrates (300 kbps, 700 kbps, 1.5 Mbps, 3 Mbps, 6 Mbps, 12 Mbps, etc.).

Security

Applications may require confidentiality (preventing eavesdropping), authentication (verifying endpoint identity), and integrity (preventing tampering). TCP and UDP alone provide no encryption, endpoint authentication, or message integrity protection. Security is added via TLS over TCP (HTTPS) or QUIC, which integrates transport and TLS security over UDP (all QUIC connections are encrypted by design).

Transport Service Requirements for Common Applications

File transfer/download, email and Web documents require no loss, elastic throughput, and are not time-sensitive.

Real-time audio/video is loss-tolerant, requires 5 Kbps-1 Mbps for audio and 10 Kbps-5 Mbps for video, and is time-sensitive (tens of milliseconds).

Streaming audio/video is loss-tolerant, has similar throughput requirements, and is time-sensitive (few seconds).

Interactive games are loss-tolerant, require Kbps+ throughput, and are time-sensitive (tens of milliseconds).

Text messaging requires no loss, elastic throughput, and has mixed time sensitivity.

Internet Transport Protocol Services

TCP provides reliable transport between sending and receiving processes, flow control (sender will not overwhelm receiver), congestion control (throttle sender when network is overloaded), and connection-oriented setup required between client and server processes. TCP does not provide timing, minimum throughput guarantees, or security.

UDP provides unreliable data transfer between sending and receiving processes. UDP does not provide reliability, flow control, congestion control, timing, throughput guarantees, security, or connection setup.

Securing TCP

Vanilla TCP and UDP sockets provide no encryption; cleartext passwords sent into sockets traverse the Internet in cleartext. Transport Layer Security (TLS) provides encrypted TCP connections, data integrity, and endpoint authentication. TLS is implemented in the application layer. Applications use TLS libraries, which use TCP in turn. Cleartext sent into a TLS socket traverses the Internet encrypted.

---

6. Web and HTTP

6.1 HTTP Protocol Basics

A web page consists of objects, each of which can be stored on different web servers. Objects can be HTML files, JPEG images, Java applets, audio files, etc. A web page consists of a base HTML file that includes several referenced objects, each addressable by a URL consisting of a hostname and path name.

HTTP Overview

HTTP (Hypertext Transfer Protocol) is the Web's application-layer protocol. It uses a client/server model: the client (browser) requests, receives (using HTTP protocol), and displays web objects, while the server (web server) sends objects in response to requests using HTTP protocol.

HTTP uses TCP. The client initiates a TCP connection (creates socket) to the server on port 80. The server accepts the TCP connection from the client. HTTP messages (application-layer protocol messages) are exchanged between browser (HTTP client) and web server (HTTP server). The TCP connection is then closed.

HTTP is stateless: the server maintains no information about past client requests. Protocols that maintain state are complex because past history (state) must be maintained, and if the server or client crashes, their views of state may be inconsistent and must be reconciled.

HTTP Connection Types

Non-persistent HTTP opens a TCP connection, sends at most one object over the TCP connection, then closes the connection. Downloading multiple objects requires multiple connections.

Persistent HTTP opens a TCP connection to a server, sends multiple objects over the single TCP connection between client and server, then closes the connection.

Non-persistent HTTP Response Time

RTT (Round Trip Time) is the time for a small packet to travel from client to server and back. HTTP response time per object is one RTT to initiate the TCP connection, one RTT for the HTTP request and first few bytes of HTTP response to return, plus object/file transmission time. Total non-persistent HTTP response time is 2RTT plus file transmission time.

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues include requiring 2 RTTs per object, OS overhead for each TCP connection, and browsers often opening multiple parallel TCP connections to fetch referenced objects in parallel.

Persistent HTTP (HTTP 1.1) leaves the connection open after sending the response. Subsequent HTTP messages between the same client/server are sent over the open connection. The client sends requests as soon as it encounters a referenced object. This requires as little as one RTT for all referenced objects, cutting response time in half.

HTTP Request Message

There are two types of HTTP messages: request and response. HTTP request messages use ASCII (human-readable format). The message consists of a request line (GET, POST, HEAD commands), header lines (Host, User-Agent, Accept, Accept-Language, Accept-Encoding, Connection, etc.), and a carriage return/line feed at the start of a line indicating the end of header lines.

The general format includes a request line with method, URL, version, followed by header field name/value pairs, a blank line (CRLF), and an entity body.

HTTP Request Methods

POST method is used when a web page includes form input. User input is sent from client to server in the entity body of the HTTP POST request message.

GET method for sending data to server includes user data in the URL field of the HTTP GET request message (following a '?').

HEAD method requests headers only that would be returned if the specified URL were requested with an HTTP GET method.

PUT method uploads a new file (object) to the server, completely replacing the file that exists at the specified URL with content in the entity body of the PUT HTTP request message.

HTTP Response Message

The response includes a status line (protocol, status code, status phrase), header lines (Date, Server, Last-Modified, ETag, Accept-Ranges, Content-Length, Content-Type), a blank line, and data (the requested HTML file).

HTTP Response Status Codes

Status codes appear in the first line of server-to-client response messages. Examples include: - 200 OK: request succeeded, requested object later in this message - 301 Moved Permanently: requested object moved, new location specified later in message (in Location field) - 400 Bad Request: request message not understood by server - 404 Not Found: requested document not found on this server - 505 HTTP Version Not Supported

Multiple Connections from Same Client

For TCP, when one client opens two browsing tabs to the same server, each tab uses a different client port number, creating separate TCP connections identified by different 4-tuples (same server IP and port, same client IP, but different client ports).

6.2 HTTP State Management and Optimization

Maintaining State: Cookies

HTTP GET/response interaction is stateless. There is no notion of multi-step exchanges to complete a web transaction, no need for client/server to track state of multi-step exchanges, all HTTP requests are independent, and no need to recover from partially-completed-but-never-completely-completed transactions.

Websites and client browsers use cookies to maintain some state between transactions. Four components are involved: cookie header line in HTTP response message, cookie header line in next HTTP request message, cookie file kept on user's host (managed by user's browser), and backend database at the website.

When initial HTTP requests arrive at a site, the site creates a unique ID (cookie) and an entry in the backend database for that ID. Subsequent HTTP requests from the user to this site contain the cookie ID value, allowing the site to identify the user.

Cookie Uses and Privacy

Cookies can be used for authorization, shopping carts, recommendations, and user session state (web email). Cookies permit sites to learn a lot about users on their site. Third-party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple websites.

The challenge is keeping state at protocol endpoints by maintaining state at sender/receiver over multiple transactions, or in messages where cookies in HTTP messages carry state.

First-party cookies are from the website you chose to visit (provides base HTML file). Third-party cookies are from websites you did not choose to visit. Tracking may be invisible to the user through invisible links rather than displayed ads.

GDPR and Cookies

The EU General Data Protection Regulation (GDPR) states that when cookies can identify an individual, cookies are considered personal data subject to GDPR personal data regulations. Users have explicit control over whether cookies are allowed.

Web Caches (Proxy Servers)

Users configure browsers to point to a local web cache. The browser sends all HTTP requests to the cache. If the object is in the cache, the cache returns the object to the client. Otherwise, the cache requests the object from the origin server, caches the received object, then returns the object to the client.

The web cache acts as both client and server: server for the original requesting client, and client to the origin server. Web caching reduces response time for client requests (cache is closer to client), reduces traffic on an institution's access link, and enables poor content providers to more effectively deliver content. The Internet is dense with caches. Servers tell caches about object's allowable caching in response headers.

Browser Caching: Conditional GET

The goal is to avoid sending the object if the browser has an up-to-date cached version, eliminating object transmission delay and network resource usage. The client specifies the date of the browser-cached copy in the HTTP request using If-modified-since: . The server response contains no object if the browser-cached copy is up-to-date: HTTP/1.0 304 Not Modified.

6.3 HTTP Evolution

HTTP/2

The key goal of HTTP/2 is decreased delay in multi-object HTTP requests. HTTP/1.1 introduced multiple pipelined GETs over a single TCP connection, but the server responds in-order (FCFS: first-come-first-served scheduling). With FCFS, small objects may wait for transmission (head-of-line blocking) behind large objects. Loss recovery (retransmitting lost TCP segments) stalls object transmission.

HTTP/2 (RFC 7540, 2015) increased flexibility at the server in sending objects to client. Methods, status codes, and most header fields remain unchanged from HTTP 1.1. Transmission order of requested objects is based on client-specified object priority (not necessarily FCFS). The server can push unrequested objects to the client and divide objects into frames, scheduling frames to mitigate HOL blocking.

HTTP/2 divides objects into frames and interleaves frame transmission. Objects can be delivered quickly even if one is slightly delayed, unlike HTTP 1.1 where objects wait behind large objects in strict order.

HTTP/2 to HTTP/3

HTTP/2 over a single TCP connection means recovery from packet loss still stalls all object transmissions. As in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling and increase overall throughput. HTTP/2 provides no security over vanilla TCP connection.

HTTP/3 adds security and per-object error and congestion control (more pipelining) over UDP. It is an application-layer protocol on top of UDP designed to increase HTTP performance, deployed on many Google servers and apps.

QUIC: Quick UDP Internet Connections

QUIC adopts approaches for connection establishment, error control, and congestion control. Error and congestion control algorithms parallel well-known TCP ones. Connection establishment achieves reliability, congestion control, authentication, encryption, and state establishment in one RTT.

For connection establishment, TCP requires a TCP handshake (transport layer) and a TLS handshake (security) in two serial handshakes before data can be sent. QUIC combines reliability, congestion control, authentication, and crypto state in a single 1 RTT handshake.

QUIC also supports 0-RTT connection establishment. The server sends a TLS session ticket that is cached at the client at the end of the first QUIC connection. At the start of a second QUIC connection, the client uses the last session ticket for encryption and authentication for the new connection, allowing data to be sent with 0 RTT handshake delay.

---

7. Electronic Mail

Electronic mail has three major components: user agents, mail servers, and the Simple Mail Transfer Protocol (SMTP).

User Agents

User agents, also known as mail readers, handle composing, editing, and reading mail messages. Examples include Outlook and iPhone mail client. Outgoing and incoming messages are stored on the server.

Mail Servers

Mail servers contain the user mailbox with incoming messages and the message queue of outgoing (to be sent) mail messages. The SMTP protocol between mail servers sends email messages. The client is the sending mail server, and the server is the receiving mail server.

SMTP Protocol

SMTP (RFC 5321) uses TCP to reliably transfer email messages from client (mail server initiating connection) to server on port 25. It provides direct transfer from sending server (acting like client) to receiving server.

Three phases occur: SMTP handshaking (greeting), SMTP transfer of messages, and SMTP closure. The protocol uses command/response interaction (like HTTP) with commands in ASCII text and responses with status code and phrase.

When Alice sends email to Bob: (1) Alice uses her user agent to compose an email message to bob@someschool.edu. (2) Alice's user agent sends the message to her mail server using SMTP; the message is placed in the message queue. (3) The client side of SMTP at the mail server opens a TCP connection with Bob's mail server. (4) The SMTP client sends Alice's message over the TCP connection. (5) Bob's mail server places the message in Bob's mailbox. (6) Bob invokes his user agent to read the message.

SMTP Observations

SMTP uses persistent connections. SMTP requires the message (header and body) to be in 7-bit ASCII. The SMTP server uses CRLF.CRLF to determine the end of the message.

Comparison with HTTP: HTTP is client pull, SMTP is client push. Both have ASCII command/response interaction and status codes. HTTP encapsulates each object in its own response message, while SMTP sends multiple objects in a multipart message.

Mail Message Format

SMTP is the protocol for exchanging email messages, defined in RFC 5321 (like RFC 7231 defines HTTP). RFC 2822 defines syntax for the email message itself (like HTML defines syntax for web documents).

The message consists of header lines (To, From, Subject) that are different from SMTP MAIL FROM and RCPT TO commands, a blank line, and the body containing the message in ASCII characters only.

Retrieving Email

SMTP handles delivery/storage of email messages to the receiver's server. Mail access protocols retrieve messages from the server. IMAP (Internet Mail Access Protocol, RFC 3501) stores messages on the server and provides retrieval, deletion, and folders of stored messages on the server. HTTP (gmail, Hotmail, Yahoo Mail, etc.) provides a web-based interface on top of SMTP (to send) and IMAP or POP (to retrieve) email messages.

---

8. Domain Name System

8.1 DNS Architecture and Services

The Domain Name System (DNS) maps between IP addresses and names. People have many identifiers (SSN, name, passport number). Internet hosts and routers have IP addresses (32 bit) used for addressing datagrams and names (such as cs.umass.edu) used by humans.

DNS is a distributed database implemented in a hierarchy of many name servers. It is an application-layer protocol where hosts and DNS servers communicate to resolve names (address/name translation). This core Internet function is implemented as an application-layer protocol, placing complexity at the network's edge.

DNS Services

DNS provides hostname-to-IP-address translation, host aliasing (canonical and alias names), mail server aliasing, and load distribution where replicated web servers have many IP addresses corresponding to one name.

Why Not Centralize DNS

Centralization does not scale due to single point of failure, traffic volume, distant centralized database, and maintenance issues. Comcast DNS servers alone handle 600 billion DNS queries per day. Akamai DNS servers alone handle 2.2 trillion DNS queries per day.

Thinking About DNS

DNS is a humongous distributed database with approximately one billion records, each simple. It handles many trillions of queries per day with many more reads than writes. Performance matters because almost every Internet transaction interacts with DNS where milliseconds count. It is organizationally and physically decentralized with millions of different organizations responsible for their records. It must be bulletproof in terms of reliability and security.

DNS Hierarchical Database

When a client wants the IP address for www.amazon.com, the client queries the root server to find the .com DNS server, queries the .com DNS server to get the amazon.com DNS server, then queries the amazon.com DNS server to get the IP address for www.amazon.com.

The hierarchy consists of root DNS servers at the top, Top Level Domain (TLD) DNS servers for .com, .org, .edu, etc., and authoritative DNS servers for specific organizations like nyu.edu, umass.edu, yahoo.com, amazon.com, pbs.org, etc.

Root Name Servers

Root name servers are the official contact of last resort for name servers that cannot resolve names. They are incredibly important; the Internet could not function without them. DNSSEC provides security (authentication, message integrity). There are 13 logical root name servers worldwide, each replicated many times. ICANN (Internet Corporation for Assigned Names and Numbers) manages the root DNS domain.

Top-Level Domain and Authoritative Servers

TLD servers are responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains (such as .cn, .uk, .fr, .ca, .jp). Network Solutions is the authoritative registry for .com and .net TLD. Educause manages the .edu TLD.

Authoritative DNS servers are the organization's own DNS servers providing authoritative hostname-to-IP mappings for the organization's named hosts. They can be maintained by the organization or a service provider.

Local DNS Name Servers

When a host makes a DNS query, it is sent to its local DNS server. The local DNS server returns a reply, answering from its local cache of recent name-to-address translation pairs (possibly out of date) or forwarding the request into the DNS hierarchy for resolution. Each ISP has a local DNS name server. The local DNS server does not strictly belong to the hierarchy.

8.2 DNS Operation and Security

DNS Name Resolution: Iterated Query

In iterated query, the contacted server replies with the name of the server to contact, essentially saying "I don't know this name, but ask this server." The query process involves the requesting host contacting the local DNS server, which then contacts the root DNS server, TLD DNS server, and authoritative DNS server in sequence, each providing the next server to contact.

DNS Name Resolution: Recursive Query

In recursive query, the burden of name resolution is placed on the contacted name server. Each server in the chain performs the full resolution before returning the result. This can create heavy load at upper levels of the hierarchy.

Caching DNS Information

Once any name server learns a mapping, it caches the mapping and immediately returns a cached mapping in response to a query. Caching improves response time. Cache entries timeout (disappear) after some time (TTL). TLD servers are typically cached in local name servers. Cached entries may be out of date; if a named host changes IP address, it may not be known Internet-wide until all TTLs expire. This provides best-effort name-to-address translation.

DNS Records

DNS is a distributed database storing resource records (RR) in the format (name, value, type, ttl).

Type A: name is hostname, value is IP address. Type NS: name is domain (such as foo.com), value is hostname of authoritative name server for this domain. Type CNAME: name is alias name for some canonical (the real) name (such as www.ibm.com is really servereast.backup2.ibm.com), value is canonical name. Type MX: value is name of SMTP mail server associated with name.

DNS Protocol Messages

DNS query and reply messages have the same format. The message header includes identification (16 bit number for query; reply to query uses same number) and flags indicating query or reply, recursion desired, recursion available, and whether the reply is authoritative. The header also includes counts for questions, answer RRs, authority RRs, and additional RRs.

The message body contains questions (variable number of questions with name and type fields for a query), answers (RRs in response to query), authority (records for authoritative servers), and additional information (additional helpful information that may be used).

Getting Information Into DNS

For a new startup (such as Network Utopia), register the name networkuptopia.com at a DNS registrar (such as Network Solutions). Provide names and IP addresses of authoritative name server (primary and secondary). The registrar inserts NS and A RRs into the .com TLD server. Create an authoritative server locally with type A record for www.networkuptopia.com and type MX record for networkuptopia.com.

DNS Security

DDoS attacks can bombard root servers with traffic (not successful to date due to traffic filtering and local DNS servers caching IPs of TLD servers, allowing root server bypass) or bombard TLD servers (potentially more dangerous).

Spoofing attacks involve intercepting DNS queries and returning bogus replies, or DNS cache poisoning. RFC 4033 defines DNSSEC authentication services.

---

9. Network Security Fundamentals

9.1 Security Concepts and Threat Model

Key Security Concepts

Security is not just a product (such as a firewall); it is a process that needs to be managed properly. Nothing is 100 percent secure. The security of a system is equivalent to the security of its least secure component (rule of the weakest link). Security by obscurity never works. Cryptography is a powerful tool but is not enough by itself. Do not rely on users to make secure decisions; users will prioritize convenience over security.

A network or system is secure when it provides appropriate protection against identified threats and vulnerabilities.

What Network Security Means

Confidentiality: only sender and intended receiver should understand message contents. The sender encrypts the message, and the receiver decrypts it.

Authentication: sender and receiver want to confirm the identity of each other.

Message integrity: sender and receiver want to ensure the message is not altered in transit or afterwards without detection.

Access and availability: services must be accessible and available to users.

Threat Model: Alice, Bob, and Trudy

Alice and Bob (lovers) want to communicate securely. Trudy (intruder) may intercept, delete, or add messages. In real life, Bob and Alice can be web browser/server for electronic transactions, online banking client/server, DNS servers, BGP routers exchanging routing table updates, or other entities.

What an Attacker Can Do

An attacker can eavesdrop (intercept messages), actively insert messages into connections, perform impersonation (fake/spoof source address in packets or any field in packets), hijack ongoing connections by removing sender or receiver and inserting themselves in place, and perform denial of service attacks to prevent service from being used by others (such as by overloading resources).

9.2 Cryptographic Foundations

The Language of Cryptography

The plaintext message is m. Ciphertext encrypted with key K_A is K_A(m). Decryption recovers the plaintext: m = K_B(K_A(m)). Alice's encryption key and Bob's decryption key are used with encryption and decryption algorithms.

Breaking Encryption Schemes

Cipher-text only attack: Trudy has ciphertext to analyze using brute force (searching through all keys) or statistical analysis.

Known-plaintext attack: Trudy has plaintext corresponding to ciphertext. In monoalphabetic cipher, Trudy can determine pairings for specific letters.

Chosen-plaintext attack: Trudy can get ciphertext for chosen plaintext.

Symmetric Key Cryptography

Bob and Alice share the same (symmetric) key K_S. The key might be knowing the substitution pattern in monoalphabetic substitution cipher. A question arises: how do Bob and Alice agree on the key value?

A simple encryption scheme uses substitution cipher, substituting one thing for another. Monoalphabetic cipher substitutes one letter for another. The encryption key is a mapping from the set of 26 letters to the set of 26 letters.

A more sophisticated approach uses n substitution ciphers M_1, M_2, …, M_n in a cycling pattern. For each new plaintext symbol, use the subsequent substitution pattern in the cyclic pattern. The encryption key is n substitution ciphers and the cyclic pattern, which need not be just an n-bit pattern.

Block Cipher

In one pass through a block cipher, one input bit affects eight output bits. With multiple passes, each input bit affects all output bits. Block ciphers include DES, 3DES, and AES.

Cipher Block Chaining

With simple block cipher, if an input block is repeated, it produces the same ciphertext. Cipher block chaining XORs the i-th input block m(i) with the previous block of ciphertext c(i-1). c(0) is transmitted to the receiver in clear. This prevents repeated input blocks from producing identical ciphertext.

DES: Data Encryption Standard

DES is the US encryption standard (NIST 1993) using a 56-bit symmetric key and 64-bit plaintext input. It is a block cipher with cipher block chaining. The security of DES is limited: the DES Challenge decrypted a 56-bit-key-encrypted phrase using brute force in less than a day. No known good analytic attack exists. To make DES more secure, 3DES encrypts three times with three different keys.

DES operation involves initial permutation, 16 identical rounds of function application (each using different 48 bits of key), and final permutation.

AES: Advanced Encryption Standard

AES is a symmetric-key NIST standard that replaced DES in November 2001. It processes data in 128-bit blocks with 128, 192, or 256-bit keys. Brute force decryption (trying each key) that takes 1 second on DES would take 149 trillion years for AES.

Public Key Cryptography

Symmetric key crypto requires sender and receiver to know a shared secret key, raising the question of how to agree on the key in the first place, particularly if they have never met.

Public key crypto is a radically different approach where sender and receiver do not share a secret key. The public encryption key is known to all, while the private decryption key is known only to the receiver. This revolutionized 2000-year-old (previously only symmetric key) cryptography. Similar ideas emerged at roughly the same time, independently in the US and UK (the UK work was classified).

Public Key Encryption Requirements

Two keys are needed: K_B^+ and K_B^- such that $K_B^{-(K_B}+(m)) = m$. Given public key K_B^+, it should be impossible to compute private key K_B^-.

RSA Algorithm

RSA (Rivest, Shamir, Adelson) is the primary public key algorithm. The message is just a bit pattern that can be uniquely represented by an integer number. Encrypting a message is equivalent to encrypting a number, which gives a new number (the ciphertext).

To create public/private key pair: (1) Choose two large prime numbers p and q (such as 1024 bits each). (2) Compute n = pq and z = (p-1)(q-1). (3) Choose e (with e < n) that has no common factors with z (e and z are relatively prime). (4) Choose d such that ed-1 is exactly divisible by z (in other words, ed mod z = 1). (5) The public key is (n,e), and the private key is (n,d).

To encrypt message m (where m < n), compute c = m^e mod n. To decrypt received bit pattern c, compute m = c^d mod n. The magic is that m = (m^e mod n)^d mod n.

RSA works because for any x and y: x^y mod n = x^(y mod z) mod n, where n = pq and z = (p-1)(q-1). Thus c^d mod n = (m^e mod n)^d mod n = m^ed mod n = m^(ed mod z) mod n = m^1 mod n = m.

RSA Important Property

An important property for later use: $K_B^{-(K_B}+(m)) = K_B^{+(K_B}-(m)) = m$. Using the public key first followed by the private key produces the same result as using the private key first followed by the public key. This follows from modular arithmetic: (m^e mod n)^d mod n = m^ed mod n = m^de mod n = (m^d mod n)^e mod n.

RSA Security

If you know Bob's public key (n,e), determining d essentially requires finding the factors of n without knowing the two factors p and q. Factoring a big number is computationally hard.

RSA in Practice: Session Keys

Exponentiation in RSA is computationally intensive. DES is at least 100 times faster than RSA. The solution is to use public key crypto to establish a secure connection, then establish a second symmetric session key K_S for encrypting data. Bob and Alice use RSA to exchange a symmetric session key K_S. Once both have K_S, they use symmetric key cryptography.

---

10. Authentication and Message Integrity

10.1 Authentication Protocols

Protocol ap1.0

Alice says "I am Alice." Failure scenario: In a network, Bob cannot see Alice, so Trudy simply declares herself to be Alice.

Protocol ap2.0

Alice says "I am Alice" in an IP packet containing her source IP address. Failure scenario: Trudy can create a packet spoofing Alice's address.

Protocol ap3.0

Alice says "I am Alice" and sends her secret password to prove it. Failure scenario: Playback attack where Trudy records Alice's packet and later plays it back to Bob.

Protocol ap3.1 (Modified ap3.0)

Alice says "I am Alice" and sends her encrypted secret password to prove it. Failure scenario: Playback attack still works because Trudy records Alice's packet and later plays it back to Bob.

Protocol ap4.0

To avoid playback attack, Bob sends Alice a nonce R (number used only once-in-a-lifetime). Alice must return R encrypted with a shared secret key. Bob knows Alice is live, and only Alice knows the key to encrypt the nonce, so it must be Alice. However, this requires a shared symmetric key.

Protocol ap5.0

Using public key techniques, Bob sends a nonce R to Alice. Alice returns K_A^-(R) (R encrypted with her private key). Bob sends Alice his public key K_A^+ and computes $K_A^{+(K_A}-(R)) = R$, knowing only Alice could have the private key that encrypted R such that $K_A^{+(K_A}-(R)) = R$.

Flaw in ap5.0: Man (or woman) in the middle attack where Trudy poses as Alice to Bob and as Bob to Alice. Trudy intercepts messages, uses her own key pair, and can decrypt messages intended for Alice.

10.2 Digital Signatures and Certification

Digital Signatures

Digital signatures are a cryptographic technique analogous to hand-written signatures. The sender (Bob) digitally signs a document, establishing he is the document owner/creator. Signatures are verifiable and nonforgeable: the recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed the document.

A simple digital signature for message m: Bob signs m by encrypting with his private key K_B^-, creating signed message K_B^-(m).

When Alice receives message m with signature K_B^-(m), she verifies m was signed by Bob by applying Bob's public key K_B^+ to K_B^-(m), then checks if $K_B^{+(K_B}$-(m)) = m. If equal, whoever signed m must have used Bob's private key. Alice thus verifies that Bob signed m, no one else signed m, and Bob signed m and not m'. Non-repudiation: Alice can take m and signature K_B^-(m) to court and prove that Bob signed m.

Message Digests

Computationally expensive public-key encryption of long messages is impractical. The goal is a fixed-length, easy-to-compute digital fingerprint. A hash function H produces a fixed-size message digest (fingerprint) from a large message m.

Hash function properties: many-to-1 mapping, produces fixed-size message digest, and given message digest x, it is computationally infeasible to find m such that x = H(m).

The Internet checksum is a poor cryptographic hash function. It produces a fixed-length digest (16-bit sum) of the message and is many-to-1, but given a message with a given hash value, it is easy to find another message with the same hash value.

Digital Signature with Message Digest

Bob sends a digitally signed message by creating a hash H(m) of the message, digitally signing the hash with his private key to create K_B^-(H(m)), and sending both the message and the encrypted message digest.

Alice verifies the signature and integrity by computing H(m) from the received message, decrypting the received encrypted digest using Bob's public key K_B^+ to get H(m), and checking if the two H(m) values are equal.

Hash Function Algorithms

MD5 hash function (RFC 1321) is widely used, computing a 128-bit message digest in a 4-step process. For an arbitrary 128-bit string x, it appears difficult to construct a message m whose MD5 hash equals x.

SHA-1 is also used, a US standard (NIST, FIPS PUB 180-1) producing a 160-bit message digest.

Fixing ap5.0

The problem was that Trudy could intercept public key exchanges and substitute her own keys. The solution requires certified public keys.

Need for Certified Public Keys

Without certification, Trudy can create fraudulent scenarios. For example, Trudy creates an email order supposedly from Bob, signs it with her private key, sends it to Pizza Store, and sends the Pizza Store her public key claiming it's Bob's public key. The Pizza Store verifies the signature and delivers pizzas to Bob, who didn't order them.

Public Key Certification Authorities (CA)

A Certification Authority (CA) binds a public key to a particular entity E. The entity (person, website, router) registers its public key with CA and provides proof of identity to CA. CA creates a certificate binding identity E to E's public key. The certificate containing E's public key is digitally signed by CA, essentially saying "this is E's public key."

When Alice wants Bob's public key, she gets Bob's certificate (from Bob or elsewhere) and applies CA's public key to Bob's certificate to get Bob's public key.

A certificate contains a serial number (unique to issuer), information about certificate owner (including algorithm and key value itself), information about certificate issuer, valid dates, and a digital signature by the issuer.

11. Securing Network Protocols

11.1 Secure Email

Secure Email: Confidentiality

When Alice wants to send confidential email m to Bob: (1) Alice generates a random symmetric private key K_S. (2) Alice encrypts the message with K_S (for efficiency). (3) Alice also encrypts K_S with Bob's public key K_B^+. (4) Alice sends both K_S(m) and K_B^+(K_S) to Bob.

Bob uses his private key to decrypt and recover K_S, then uses K_S to decrypt K_S(m) to recover m.

Secure Email: Integrity and Authentication

When Alice wants to send m to Bob with message integrity and authentication: (1) Alice digitally signs a hash of her message with her private key K_A^-, providing integrity and authentication. (2) Alice sends both the message (in the clear) and the digital signature K_A^-(H(m)).

Bob computes H(m) from the received message and compares it to the decrypted signature.

Secure Email: Complete Solution

To provide confidentiality, message integrity, and authentication, Alice uses three keys: her private key, Bob's public key, and a new symmetric key. The message is hashed and signed with Alice's private key for integrity and authentication, then the message and signature are encrypted with a symmetric key for confidentiality, and the symmetric key is encrypted with Bob's public key for secure key distribution.

Pretty Good Privacy (PGP)

PGP is the de-facto standard Internet email encryption scheme. It uses symmetric key cryptography, public key cryptography, hash functions, and digital signatures as described. It provides secrecy, sender authentication, and integrity. The inventor, Phil Zimmerman, was the target of a 3-year federal investigation.

11.2 Transport Layer Security (TLS)

TLS Overview

TLS is a widely deployed security protocol above the transport layer, supported by almost all browsers and web servers using HTTPS (port 443). TLS provides confidentiality via symmetric encryption, integrity via cryptographic hashing, and authentication via public key cryptography (all techniques previously studied).

History: early research and implementation in secure network programming and secure sockets, Secure Socket Layer (SSL) deprecated in 2015, TLS 1.3 defined in RFC 8846 (2018).

TLS Requirements

Handshake: Alice and Bob use their certificates and private keys to authenticate each other and exchange or create a shared secret.

Key derivation: Alice and Bob use the shared secret to derive a set of keys.

Data transfer: stream data transfer as a series of records (not just one-time transactions).

Connection closure: special messages to securely close the connection.

t-tls: Toy TLS Handshake

In the t-tls handshake phase: Bob establishes a TCP connection with Alice, Bob verifies that Alice is really Alice, and Bob sends Alice a master secret key (MS) used to generate all other keys for the TLS session. A potential issue is 3 RTT before the client can start receiving data (including TCP handshake).

t-tls: Cryptographic Keys

It is considered bad practice to use the same key for more than one cryptographic function. Different keys are used for MAC and encryption. Four keys are needed: K_c (encryption key for data sent from client to server), M_c (MAC key for data sent from client to server), K_s (encryption key for data sent from server to client), and M_s (MAC key for data sent from server to client).

Keys are derived from a key derivation function (KDF) that takes the master secret and possibly some additional random data to create new keys.

t-tls: Encrypting Data

TCP provides a data byte stream abstraction. Data cannot be encrypted in-stream as written into the TCP socket because there would be no place for the MAC, and if the MAC were at the end, there would be no message integrity until all data is received and the connection closed.

The solution is to break the stream into a series of records. Each client-to-server record carries a MAC created using M_c. The receiver can act on each record as it arrives. The t-tls record (length, data, MAC) is encrypted using symmetric key K_c and passed to TCP.

t-tls: Sequence Numbers and Nonces

Possible attacks on the data stream include re-ordering (man-in-middle intercepts TCP segments and reorders them by manipulating sequence numbers in the unencrypted TCP header) and replay attacks. Solutions include using TLS sequence numbers (data and TLS-seq-# incorporated into MAC) and using nonces.

t-tls: Connection Close

To prevent truncation attacks where an attacker forges a TCP connection close segment making one or both sides think there is less data than actually exists, the solution uses record types with one type for closure. Type 0 is for data, type 1 is for close. MAC is now computed using data, type, and sequence number.

TLS 1.3 Cipher Suite

A cipher suite defines algorithms for key generation, encryption, MAC, and digital signature. TLS 1.3 (2018) has a more limited cipher suite choice than TLS 1.2 (2008): only 5 choices rather than 37. TLS 1.3 requires Diffie-Hellman (DH) for key exchange rather than DH or RSA. It uses combined encryption and authentication algorithm (authenticated encryption) for data rather than serial encryption and authentication. Four cipher suites are based on AES. HMAC uses SHA (256 or 384) cryptographic hash function.

TLS 1.3: 1 RTT Handshake

The client TLS hello message guesses the key agreement protocol and parameters and indicates cipher suites it supports. The server TLS hello message chooses the key agreement protocol, parameters, and cipher suite, and provides a server-signed certificate. The client checks the server certificate, generates the key, and can now make an application request (such as HTTPS GET).

TLS 1.3: 0 RTT Handshake

The initial hello message contains encrypted application data when resuming an earlier connection between client and server. Application data is encrypted using a resumption master secret from the earlier connection. This is vulnerable to replay attacks but may be acceptable for HTTP GET or client requests not modifying server state.

11.3 Network Layer Security (IPsec)

IPsec Overview

IPsec provides datagram-level encryption, authentication, and integrity for both user traffic and control traffic (such as BGP, DNS messages). It has two modes: transport mode (only datagram payload is encrypted and authenticated) and tunnel mode (entire datagram is encrypted and authenticated, then encapsulated in a new datagram with new IP header and tunneled to destination).

Two IPsec Protocols

Authentication Header (AH) protocol (RFC 4302) provides source authentication and data integrity but not confidentiality. Encapsulation Security Protocol (ESP) (RFC 4303) provides source authentication, data integrity, and confidentiality. ESP is more widely used than AH.

Security Associations (SAs)

Before sending data, a security association (SA) is established from sending to receiving entity (directional). Sending and receiving entities maintain state information about the SA. TCP endpoints also maintain state information. IP is connectionless, but IPsec is connection-oriented.

The sending entity stores for each SA: 32-bit identifier (Security Parameter Index or SPI), origin SA interface, destination SA interface, type of encryption used, encryption key, type of integrity check used, and authentication key.

IPsec Datagram

In ESP tunnel mode, the format includes: new IP header, ESP header (SPI, Seq #), original IP header, original IP datagram payload, ESP trailer (padding, pad length, next header), and ESP auth. The encrypted portion includes the original IP header through the ESP trailer. The authenticated portion includes the ESP header through the ESP trailer.

ESP trailer includes padding for block ciphers. ESP header contains SPI so the receiving entity knows what to do and a sequence number to thwart replay attacks. MAC in the ESP auth field is created with a shared secret key.

ESP Tunnel Mode Actions

At the sending router R1: (1) Append ESP trailer to original datagram (which includes original header fields). (2) Encrypt the result using the algorithm and key specified by SA. (3) Append ESP header to the front of this encrypted quantity. (4) Create authentication MAC using the algorithm and key specified in SA. (5) Append MAC to form payload. (6) Create new IP header with new IP header fields and addresses to tunnel endpoint.

IPsec Sequence Numbers

For a new SA, the sender initializes the sequence number to 0. Each time a datagram is sent on SA, the sender increments the sequence number counter and places the value in the sequence number field. The goal is to prevent an attacker from sniffing and replaying a packet; receipt of duplicate authenticated IP packets may disrupt service. The destination checks for duplicates without keeping track of all received packets, instead using a window.

Security Policy Database (SPD) and Security Association Database (SAD)

SPD defines "what" to do: For a given datagram, the sender needs to know if it should use IPsec. Policy is stored in the SPD and may use source and destination IP address and protocol number.

SAD defines "how" to do it: The endpoint holds SA state in the SAD. When sending an IPsec datagram, R1 accesses SAD to determine how to process the datagram. When an IPsec datagram arrives at R2, R2 examines the SPI in the IPsec datagram, indexes SAD with the SPI, and processes the datagram accordingly.

IPsec Services Summary

Trudy sits somewhere between R1 and R2 and doesn't know the keys. She will not be able to see original contents of the datagram or flip bits without detection or masquerade as R1 using R1's IP address or replay a datagram.

IKE: Internet Key Exchange

Manual establishment of IPsec SAs in IPsec endpoints is impractical for VPNs with hundreds of endpoints. IPsec IKE (Internet Key Exchange) provides automated keying.

Authentication uses either pre-shared secret (PSK) or PKI (public/private keys and certificates). With PSK, both sides start with a secret and run IKE to authenticate each other and generate IPsec SAs (one in each direction), including encryption and authentication keys. With PKI, both sides start with public/private key pairs and certificates, run IKE to authenticate each other, and obtain IPsec SAs (one in each direction), similar to the handshake in SSL.

IKE has two phases: Phase 1 establishes a bi-directional IKE SA (also called ISAKMP security association, different from IPsec SA). Phase 2 uses ISAKMP to securely negotiate an IPsec pair of SAs. Phase 1 has two modes: aggressive mode (uses fewer messages) and main mode (provides identity protection and is more flexible).

IPsec Summary

IPsec involves IKE message exchange for algorithms, secret keys, and SPI numbers. Either AH or ESP protocol (or both) can be used. AH provides integrity and source authentication. ESP protocol (with AH) additionally provides encryption. IPsec peers can be two end systems, two routers/firewalls, or a router/firewall and an end system.

11.4 Wireless and Mobile Security

802.11 Authentication and Encryption

An arriving mobile device must associate with an access point (establish communication over wireless link) and authenticate to the network.

The process has four steps: (1) Discovery of security capabilities where the AP advertises its presence and forms of authentication and encryption provided, and the device requests specific forms of authentication and encryption desired. (2) Mutual authentication and shared symmetric key derivation where AS and mobile already have a shared common secret (such as a password). AS and mobile use the shared secret, nonces (to prevent relay attacks), and cryptographic hashing (to ensure message integrity) to authenticate each other. AS and mobile derive a symmetric session key. (3) Shared symmetric session key distribution where the same key is derived at mobile and AS, and AS informs the AP of the shared symmetric session. (4) Encrypted communication between mobile and remote host via AP.

802.11 WPA3 Handshake

The AS generates Nonce_AS and sends it to mobile. Mobile receives Nonce_AS, generates Nonce_M, generates symmetric shared session key K_M-AP using Nonce_AS, Nonce_M, and initial shared secret, and sends Nonce_M and HMAC-signed value using Nonce_AS and initial shared secret. AS derives symmetric shared session key K_M-AP using initial shared secret, Nonce_AS, and Nonce_M.

802.11 Protocol Stack

Extensible Authentication Protocol (EAP) (RFC 3748) defines an end-to-end request/response protocol between mobile device and AS. The protocol stack uses EAP TLS between mobile and AS, EAP over LAN (EAPoL) between mobile and AP, IEEE 802.11 for wireless, and RADIUS over UDP/IP between AP and AS.

Authentication and Encryption in 4G LTE

An arriving mobile must associate with a base station (establish communication over 4G wireless link) and authenticate itself to the network while the network authenticates itself to the mobile.

Notable differences from WiFi: the mobile's SIM card provides global identity and contains shared keys, and services in the visited network depend on paid service subscriptions in the home network.

The mobile and BS use derived session key K_BS-M to encrypt communications over the 4G link. MME in the visited network and HSS in the home network together play the role of WiFi AS. The ultimate authenticator is HSS. There is trust and a business relationship between visited and home networks.

4G LTE Authentication Process

(a) Authentication request to home network: Mobile sends attach message containing its IMSI and visited network info, relayed from BS to visited MME to home HSS. IMSI identifies the mobile's home network.

(b) HSS uses shared-in-advance secret key K_HSS-M to derive authentication token (auth_token) and expected authentication response token (xres_HSS). Auth_token contains info encrypted by HSS using K_HSS-M, allowing mobile to know that whoever computed auth_token knows the shared-in-advance secret. Mobile has authenticated the network. Visited HSS keeps xres_HSS for later use.

(c) Authentication response from mobile: Mobile computes res_M using its secret key to make the same cryptographic calculation that HSS made to compute xres_HSS and sends res_M to MME.

(d) Mobile is authenticated by network: MME compares mobile-computed value of res_M with HSS-computed value of xres_HSS. If they match, mobile is authenticated. MME informs BS that mobile is authenticated and generates keys for BS.

(e) Key derivation: Mobile and BS determine keys for encrypting data and control frames over 4G wireless channel. AES can be used.

From 4G to 5G Authentication

Key differences: 4G has MME in visited network making the authentication decision, while 5G has the home network providing the authentication decision (visited MME plays middleman role but can still reject). 4G uses shared-in-advance keys, while 5G does not share keys in advance for IoT. 4G transmits device IMSI in cleartext to BS, while 5G uses public key crypto to encrypt IMSI.

---

12. Operational Security

Firewalls Overview

Firewalls isolate an organization's internal network from the larger Internet, allowing some packets to pass while blocking others. The administered network contains trusted "good guys," while the public Internet contains untrusted "bad guys."

Why Firewalls

Firewalls prevent denial of service attacks (such as SYN flooding where attackers establish many bogus TCP connections leaving no resources for real connections), prevent illegal modification/access of internal data (such as an attacker replacing a website's homepage), and allow only authorized access to the inside network (a set of authenticated users/hosts).

Three Types of Firewalls

The three types are stateless packet filters, stateful packet filters, and application gateways.

Stateless Packet Filtering

The internal network connects to the Internet via a router firewall that filters packet-by-packet. The decision to forward/drop packets is based on source IP address, destination IP address, TCP/UDP source and destination port numbers, ICMP message type, and TCP SYN and ACK bits.

Example 1: Block incoming and outgoing datagrams with IP protocol field = 17 and with either source or destination port = 23. Result: all incoming and outgoing UDP flows and telnet connections are blocked.

Example 2: Block inbound TCP segments with ACK=0. Result: prevents external clients from making TCP connections with internal clients but allows internal clients to connect to outside.

Stateless Packet Filtering Policy Examples

No outside Web access: drop all outgoing packets to any IP address, port 80.

No incoming TCP connections except to institution's public Web server only: drop all incoming TCP SYN packets to any IP except the specified address, port 80.

Prevent Web-radios from consuming bandwidth: drop all incoming UDP packets except DNS and router broadcasts.

Prevent network from being used for smurf DoS attack: drop all ICMP packets going to broadcast addresses.

Prevent network from being tracerouted: drop all outgoing ICMP TTL expired traffic.

Access Control Lists

ACLs are tables of rules applied top to bottom to incoming packets with (action, condition) pairs. This looks like OpenFlow forwarding. Rules specify action, source address, destination address, protocol, source port, destination port, and flag bits. A typical final rule is to deny all traffic that doesn't match earlier rules.

Stateful Packet Filtering

Stateless packet filters are a heavy-handed tool that admits packets that make no sense, such as destination port = 80 with ACK bit set even though no TCP connection was established.

Stateful packet filters track the status of every TCP connection. They track connection setup (SYN) and teardown (FIN) to determine whether incoming and outgoing packets make sense. They timeout inactive connections at the firewall so packets are no longer admitted for those connections.

ACLs are augmented with a column to indicate the need to check the connection state table before admitting packets.

Application Gateways

Application gateways filter packets on application data as well as on IP/TCP/UDP fields. For example, to allow select internal users to telnet outside: (1) Require all telnet users to telnet through the gateway. (2) For authorized users, the gateway sets up a telnet connection to the destination host and relays data between the two connections. (3) Router filters block all telnet connections not originating from the gateway.

Limitations of Firewalls and Gateways

IP spoofing: routers can't know if data really comes from the claimed source. If multiple apps need special treatment, each requires its own app gateway. Client software must know how to contact the gateway (such as setting IP address of proxy in Web browser). Filters often use all-or-nothing policy for UDP. There is a tradeoff between degree of communication with the outside world and level of security. Many highly protected sites still suffer from attacks.

Intrusion Detection Systems

Packet filtering operates on TCP/IP headers only with no correlation check among sessions. IDS (intrusion detection system) performs deep packet inspection: looking at packet contents (such as checking character strings in packets against databases of known virus and attack strings) and examining correlation among multiple packets (port scanning, network mapping, DoS attacks).

Multiple IDSs can perform different types of checking at different locations. They are placed in a demilitarized zone between the firewall and the internal network, monitoring traffic to web servers, FTP servers, DNS servers, and the internal network.

13. Video Streaming and Content Distribution Networks

13.1 Video Streaming Fundamentals

Context and Challenge

Stream video traffic is a major consumer of Internet bandwidth. Netflix, YouTube, and Amazon Prime account for 80 percent of residential ISP traffic as of 2020. The challenges include scale (how to reach approximately 1 billion users) and heterogeneity (different users have different capabilities, such as wired versus mobile and bandwidth rich versus bandwidth poor). The solution involves distributed, application-level infrastructure.

Video Characteristics

Video is a sequence of images displayed at constant rate, such as 24 images per second. A digital image is an array of pixels, with each pixel represented by bits. Coding uses redundancy within and between images to decrease the number of bits used to encode the image through spatial coding (within image) and temporal coding (from one image to next).

Spatial coding involves sending only two values instead of N values of the same color: the color value and the number of repeated values. Temporal coding sends only the differences from one frame to the next instead of sending a complete frame.

Video encoding can use constant bit rate (CBR) where the video encoding rate is fixed, or variable bit rate (VBR) where the video encoding rate changes as the amount of spatial and temporal coding changes. Examples include MPEG 1 (CD-ROM) at 1.5 Mbps, MPEG2 (DVD) at 3-6 Mbps, and MPEG4 (often used in Internet) at 64 Kbps to 12 Mbps.

Streaming Stored Video

The main challenges are that server-to-client bandwidth will vary over time with changing network congestion levels (in house, access network, network core, video server), and packet loss and delay due to congestion will delay playout or result in poor video quality.

In the streaming timeline, video is recorded at constant rate (such as 30 frames per second), sent over the network with variable network delay, and received and played out at the client at constant rate (30 frames per second). Streaming means that at any given time, the client is playing out the early part of the video while the server is still sending the later part of the video.

Playout Buffering

The continuous playout constraint requires that during client video playout, playout timing must match original timing, but network delays are variable (jitter), necessitating a client-side buffer to match the continuous playout constraint.

Other challenges include client interactivity (pause, fast-forward, rewind, jump through video) and the possibility that video packets may be lost and retransmitted.

Client-side buffering and playout delay compensate for network-added delay and delay jitter. The buffer accumulates received data before playout begins.

DASH: Dynamic Adaptive Streaming over HTTP

On the server side, the video file is divided into multiple chunks, each chunk is encoded at multiple different rates, different rate encodings are stored in different files, files are replicated in various CDN nodes, and a manifest file provides URLs for different chunks.

On the client side, the client periodically estimates server-to-client bandwidth, consults the manifest, and requests one chunk at a time, choosing the maximum coding rate sustainable given current bandwidth. The client can choose different coding rates at different points in time (depending on available bandwidth at time) and from different servers.

Intelligence resides at the client, which determines when to request a chunk (so that buffer starvation or overflow does not occur), what encoding rate to request (higher quality when more bandwidth is available), and where to request the chunk (can request from URL server that is close to client or has high available bandwidth).

Streaming video combines encoding, DASH, and playout buffering.

13.2 Content Distribution Networks

CDN Motivation and Architecture

The challenge is how to stream content (selected from millions of videos) to hundreds of thousands of simultaneous users. Option 1 (single large mega-server) fails due to single point of failure, point of network congestion, long and possibly congested paths to distant clients. This solution does not scale.

Option 2 stores and serves multiple copies of videos at multiple geographically distributed sites (CDN). Two deployment strategies exist: enter deep (push CDN servers deep into many access networks, close to users; Akamai deploys 240,000 servers in more than 120 countries) and bring home (smaller number, tens of larger clusters in POPs near access networks; used by Limelight).

Netflix and CDN Operation

Netflix stores copies of content at its worldwide OpenConnect CDN nodes. When a subscriber requests content, the service provider returns a manifest file. Using the manifest, the client retrieves content at the highest supportable rate and may choose different rates or copies if the network path becomes congested.

Content distribution networks operate as an over-the-top (OTT) service, using Internet host-to-host communication as a service. OTT challenges include coping with a congested Internet from the edge: deciding what content to place in which CDN node and determining from which CDN node to retrieve content and at which rate.

---

14. Socket Programming

14.1 Socket Programming Fundamentals

Socket Concept

The goal is to learn how to build client/server applications that communicate using sockets. A socket is the door between application process and end-to-end transport protocol. Two socket types exist for two transport services: UDP (unreliable datagram) and TCP (reliable, byte stream-oriented).

Application Example

A typical example involves: (1) Client reads a line of characters (data) from its keyboard and sends data to server. (2) Server receives the data and converts characters to uppercase. (3) Server sends modified data to client. (4) Client receives modified data and displays line on its screen.

14.2 UDP Socket Programming

UDP Characteristics

UDP has no connection between client and server: no handshaking before sending data, sender explicitly attaches IP destination address and port number to each packet, receiver extracts sender IP address and port number from received packet. UDP provides unreliable transfer of groups of bytes (datagrams) between client and server processes. UDP transmitted data may be lost or received out-of-order.

UDP Client/Server Interaction

The server creates socket with specified port x: `serverSocket = socket(AF_INET, SOCK_DGRAM)` and reads datagrams from serverSocket. The client creates socket: `clientSocket = socket(AF_INET, SOCK_DGRAM)`, creates datagram with server IP address and port x, sends datagram via clientSocket, reads datagram from clientSocket, and closes clientSocket. The server writes reply to serverSocket specifying client address and port number.

UDP Client Implementation

The Python UDP client includes the socket library, specifies server name and port, creates UDP socket, gets user keyboard input, attaches server name and port to message and sends into socket, reads reply data (bytes) from socket, prints received string, and closes socket.

UDP Server Implementation

The Python UDP server creates UDP socket, binds socket to local port number, loops forever reading from UDP socket into message and getting client's address (client IP and port), and sends uppercase string back to the client.

14.3 TCP Socket Programming

TCP Characteristics

The client must contact the server where the server process must first be running and the server must have created a socket (door) that welcomes client's contact. The client contacts the server by creating a TCP socket specifying IP address and port number of the server process. When the client creates the socket, client TCP establishes a connection to server TCP.

When contacted by the client, server TCP creates a new socket for the server process to communicate with that particular client, allowing the server to talk with multiple clients. Client source port number and IP address are used to distinguish clients.

TCP provides reliable, in-order byte-stream transfer (pipe) between client and server processes from an application viewpoint.

TCP Client/Server Interaction

The server creates socket for incoming requests on port x: `serverSocket = socket()`, waits for incoming connection request: `connectionSocket = serverSocket.accept()`, reads request from connectionSocket, writes reply to connectionSocket, and closes connectionSocket. The client creates socket and connects to hostid on port x: `clientSocket = socket()`, sends request using clientSocket, reads reply from clientSocket, and closes clientSocket. TCP connection setup occurs between client socket creation and data exchange.

TCP Client Implementation

The Python TCP client creates TCP socket for server on remote port, connects to server (note: no need to attach server name and port), gets user input, sends data into socket, reads reply from socket, prints received string, and closes socket.

TCP Server Implementation

The Python TCP server creates TCP welcoming socket, binds to local port, begins listening for incoming TCP requests, loops forever, server waits on accept() for incoming requests with new socket created on return, reads bytes from socket (but not address as in UDP), sends reply, and closes connection to this client (but not welcoming socket).

License and Attribution

License

This work is released into the public domain under the Creative Commons Zero v1.0 Universal (CC0 1.0) license. You are free to:

Use this material for any purpose (personal, educational, commercial) Modify and create derivative works without restriction Distribute and share without asking permission No attribution required (though appreciated)

Attribution

While not legally required under CC0, attribution is appreciated:

Original Course Materials Source: Sistemi Operativi e Reti di Calcolatori (SOReCa) lecture slides Institution: Corso di Laurea in Ingegneria Informatica, Università La Sapienza Academic Year: 2025/2026

Compilation and Synthesis Compiled by: UniDevPremium Synthesized with assistance from: Claude (Anthropic) Date: January 2026 Method: Theory extraction, paraphrasing, and reorganization from original lecture materials

Disclaimer

This document is a synthesis and paraphrasing of educational materials for study purposes. It represents an interpretation and reorganization of the original course content. While efforts have been made to maintain technical accuracy, this is not an official course document. Always refer to original course materials and instructors for authoritative information. The original lecture content and intellectual property rights belong to the respective course instructors and academic institutions.